LAB 5

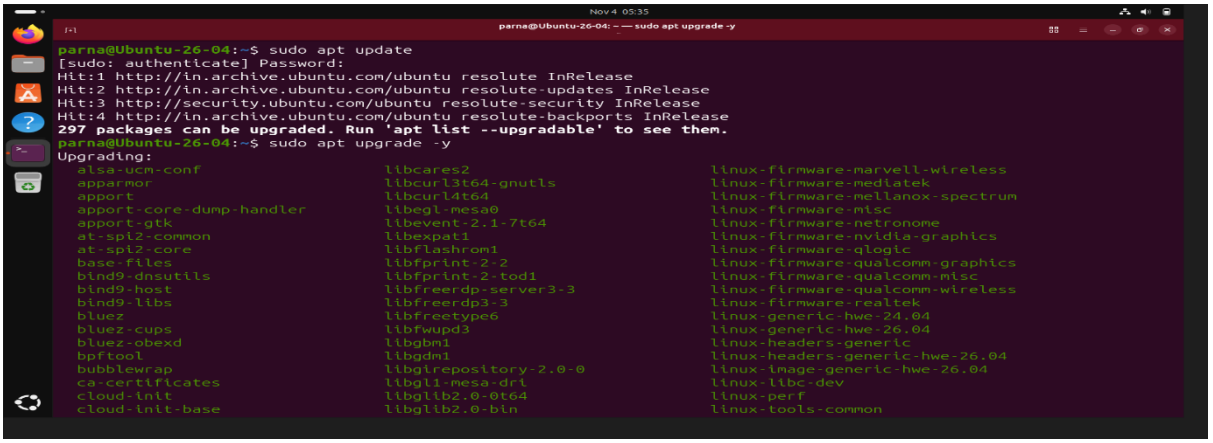
Implementation of Various Scheduling Mechanisms Using

an Open-Source Cloud Simulator

1 Update Ubuntu

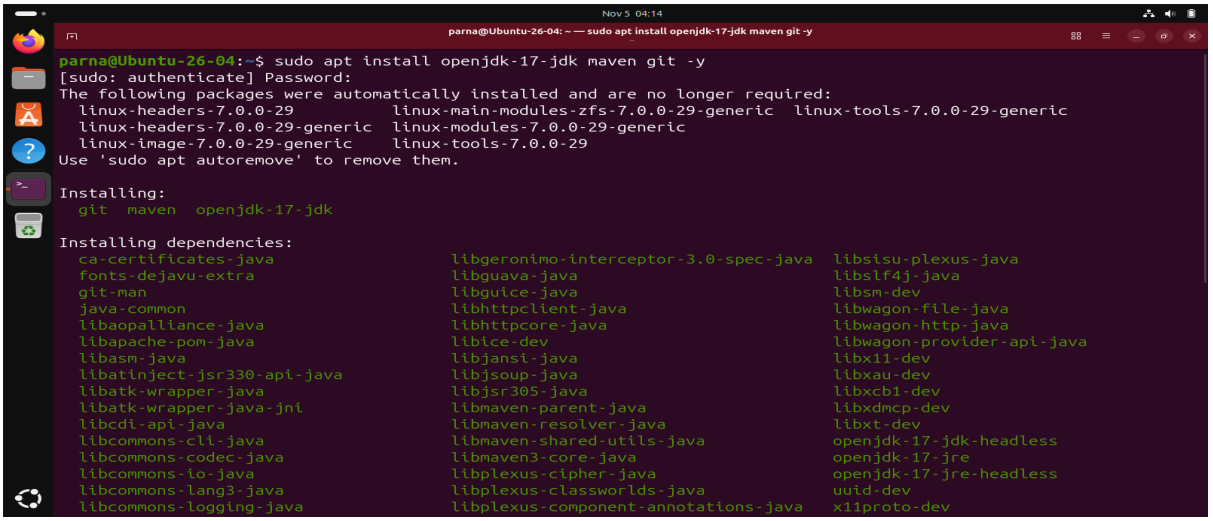
sudo apt update

sudo apt upgrade -y



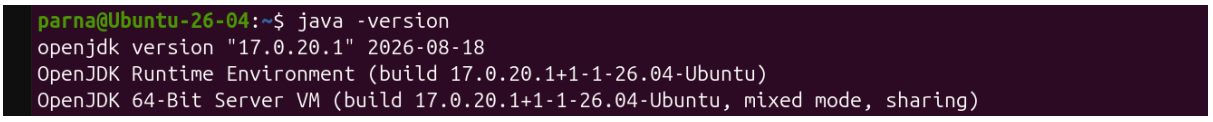
2 Install Java, Maven and Git

sudo apt install openjdk-17-jdk maven git -y



Why: Installs the Java compiler/runtime, Maven build tool and Git version-control utility.

java -version



Why: Checks the installed Java runtime version.

javac -version



Why: Checks that the Java compiler is available.

mvn -version



Why: Confirms Maven installation and displays the Java version Maven uses.

git –version



Why: Confirms Git installation.

Expected check: Java and javac should report version 17.x. Maven should display a version and show that it is using Java 17.

Resolve any Java-version mismatch before proceeding.

3. Create the CloudSim Plus Maven project

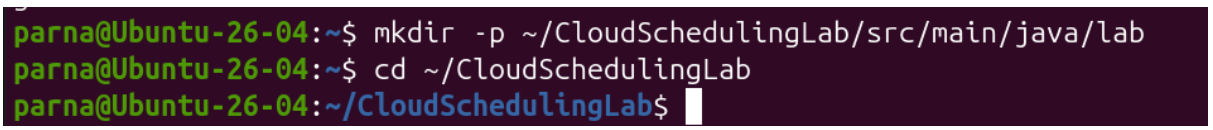
3.1 Create folders

mkdir -p ~/CloudSchedulingLab/src/main/java/lab

Why: Creates the project directory and standard Maven Java source directory. -p creates missing parent directories too.

cd ~/CloudSchedulingLab

Why: Changes the terminal's current directory to the project root.



3.2 Create pom.xml

nano pom.xml

Why: Opens the nano text editor to create the Maven configuration file.

4. Implement the experiment

nano src/main/java/lab/SchedulingExperiment.java

Why: Creates the Java source file containing the simulation setup, scheduler configurations and output code.

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         https://maven.apache.org/xsd/maven-4.0.0.xsd">

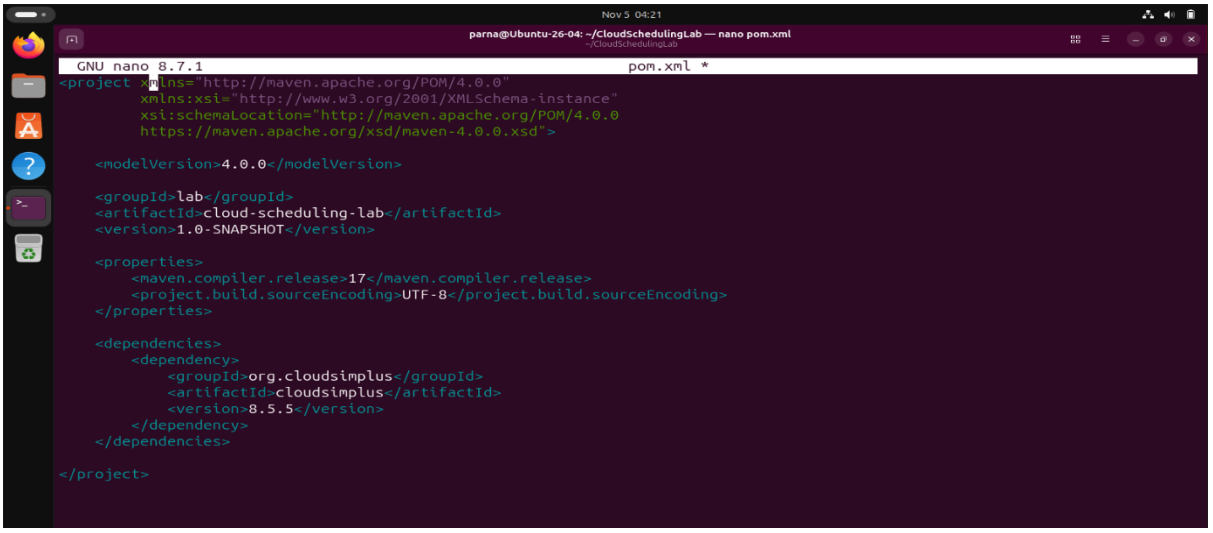
    <modelVersion>4.0.0</modelVersion>

    <groupId>lab</groupId>
    <artifactId>cloud-scheduling-lab</artifactId>
    <version>1.0-SNAPSHOT</version>

    <properties>
       <maven.compiler.release>17</maven.compiler.release>
       <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    </properties>

    <dependencies>
       <dependency>
          <groupId>org.cloudsimplus</groupId>
          <artifactId>cloudsimplus</artifactId>
          <version>8.5.5</version>
       </dependency>
    </dependencies>

</project>
```

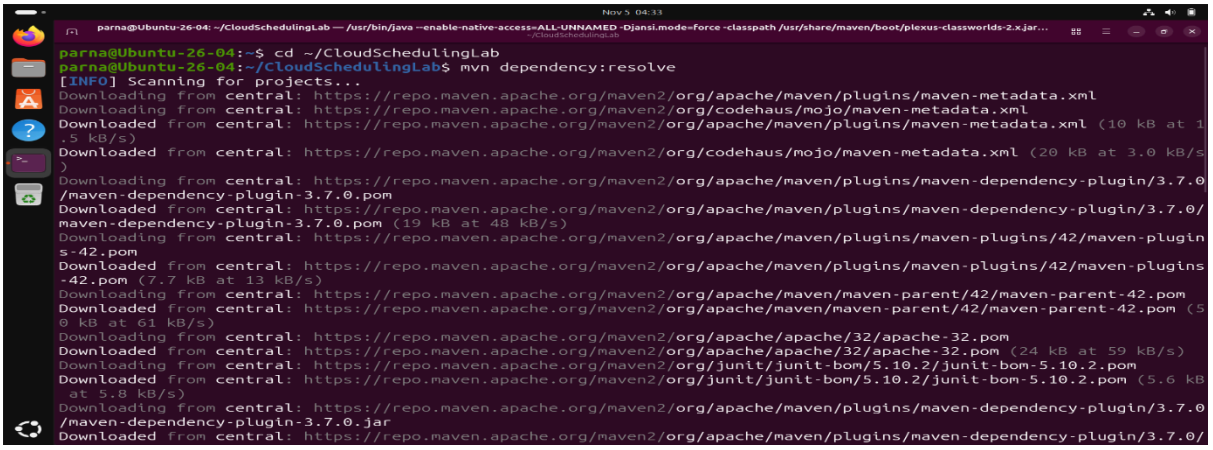


Save in nano: Ctrl+O, press Enter, then Ctrl+X.

Why this file is required: It defines the Maven project, sets Java 17 as the target and declares CloudSim Plus as a dependency. Maven downloads the required library files automatically.

3.3 Resolve dependencies

mvn dependency:resolve



Why: Downloads the declared CloudSim Plus library and supporting dependencies into Maven's local repository.

mvn clean



Why: Removes old build output (the target directory), ensuring the next build starts cleanly.

4. Implement the experiment

nano src/main/java/lab/SchedulingExperiment.java

Why: Creates the Java source file containing the simulation setup, scheduler configurations and output code.

package lab;

```
import java.util.ArrayList;
import java.util.List;

import org.cloudsimplus.core.CloudSimPlus;
import org.cloudsimplus.datacenters.Datacenter;
import org.cloudsimplus.datacenters.DatacenterSimple;
import org.cloudsimplus.brokers.DatacenterBroker;
import org.cloudsimplus.brokers.DatacenterBrokerSimple;
import org.cloudsimplus.cloudlets.Cloudlet;
import org.cloudsimplus.cloudlets.CloudletSimple;
import org.cloudsimplus.hosts.Host;
import org.cloudsimplus.hosts.HostSimple;
import org.cloudsimplus.resources.Pe;
import org.cloudsimplus.resources.PeSimple;
import org.cloudsimplus.schedulers.cloudlet.CloudletScheduler;
import org.cloudsimplus.schedulers.cloudlet.CloudletSchedulerSpaceShared;
import org.cloudsimplus.schedulers.cloudlet.CloudletSchedulerTimeShared;
import org.cloudsimplus.schedulers.vm.VmScheduler;
import org.cloudsimplus.schedulers.vm.VmSchedulerSpaceShared;
import org.cloudsimplus.schedulers.vm.VmSchedulerTimeShared;
import org.cloudsimplus.utilizationmodels.UtilizationModel;
import org.cloudsimplus.utilizationmodels.UtilizationModelFull;
import org.cloudsimplus.vms.Vm;
import org.cloudsimplus.vms.VmSimple;
import org.cloudsimplus.provisioners.ResourceProvisionerSimple;
import org.cloudsimplus.allocationpolicies.VmAllocationPolicySimple;

public class SchedulingExperiment {

    private static final int HOST_PES = 4;
    private static final int VM_COUNT = 2;
    private static final int CLOUDLET_COUNT = 4;

    private static final long MIPS = 1000;
    private static final long CLOUDLET_LENGTH = 20000;

    public static void main(String[] args) {

        // Scenario A
        runScenario(
            "A: VM SpaceShared + Cloudlet SpaceShared",
            new VmSchedulerSpaceShared(),
            new CloudletSchedulerSpaceShared()
        );

        // Scenario B
        runScenario(
            "B: VM SpaceShared + Cloudlet TimeShared",
            new VmSchedulerSpaceShared(),
            new CloudletSchedulerTimeShared()
        );

        // Scenario C
        runScenario(
            "C: VM TimeShared + Cloudlet SpaceShared",
```

```
            new VmSchedulerTimeShared(),
            new CloudletSchedulerSpaceShared()
        );

        // Scenario D
        runScenario(
            "D: VM TimeShared + Cloudlet TimeShared",
            new VmSchedulerTimeShared(),
            new CloudletSchedulerTimeShared()
        );
    }

    // Execute individual scheduling scenario
    private static void runScenario(
        String title,
        VmScheduler vmScheduler,
        CloudletScheduler cloudletScheduler) {

        System.out.println("\n================================================");
        System.out.println(title);
        System.out.println("================================================");

        CloudSimPlus simulation = new CloudSimPlus();

        createDatacenter(simulation, vmScheduler);

        DatacenterBroker broker =
            new DatacenterBrokerSimple(simulation);

        List<Vm> vmList = createVms(cloudletScheduler);

        List<Cloudlet> cloudletList = createCloudlets();

        broker.submitVmList(vmList);

        broker.submitCloudletList(cloudletList);

        simulation.start();

        List<Cloudlet> finished =
            broker.getCloudletFinishedList();

        System.out.printf(
            "%-10s %-10s %-12s %-12s %-12s%n",
            "Cloudlet", "VM", "Status", "Start", "Finish"
        );

        for (Cloudlet cloudlet : finished) {

            System.out.printf(
                "%-10d %-10d %-12s %-12.2f %-12.2f%n",
                cloudlet.getId(),
                cloudlet.getVm().getId(),
                cloudlet.getStatus(),
                cloudlet.getStartTime(),
```

```
            cloudlet.getFinishTime()
        );
    }

    System.out.println(
        "Completed Cloudlets: "
        + finished.size() + " / " + CLOUDLET_COUNT
    );
}

// Create Datacenter and Host
private static Datacenter createDatacenter(
    CloudSimPlus simulation,
    VmScheduler vmScheduler) {

    List<Pe> peList = new ArrayList<>();

    for (int i = 0; i < HOST_PES; i++) {

        peList.add(new PeSimple(MIPS));
    }

    Host host = new HostSimple(
        8192,
        100000,
        1000000,
        peList
    );

    host.setRamProvisioner(
        new ResourceProvisionerSimple()
    );

    host.setBwProvisioner(
        new ResourceProvisionerSimple()
    );

    host.setVmScheduler(vmScheduler);

    List<Host> hostList = new ArrayList<>();

    hostList.add(host);

    return new DatacenterSimple(
        simulation,
        hostList,
        new VmAllocationPolicySimple()
    );
}

// Create Virtual Machines
private static List<Vm> createVms(
    CloudletScheduler scheduler) {

    List<Vm> vmList = new ArrayList<>();
```

```
        for (int i = 0; i < VM_COUNT; i++) {

            Vm vm = new VmSimple(i, MIPS, 2);

            vm.setRam(2048)
              .setBw(10000)
              .setSize(10000);

            // Each VM must have its own scheduler instance
            CloudletScheduler vmScheduler;

            if (scheduler instanceof CloudletSchedulerSpaceShared) {

                vmScheduler =
                    new CloudletSchedulerSpaceShared();

            } else {

                vmScheduler =
                    new CloudletSchedulerTimeShared();
            }

            vm.setCloudletScheduler(vmScheduler);

            vmList.add(vm);
        }

        return vmList;
    }

    // Create Cloudlets
    private static List<Cloudlet> createCloudlets() {

        List<Cloudlet> cloudletList = new ArrayList<>();

        UtilizationModel utilization =
            new UtilizationModelFull();

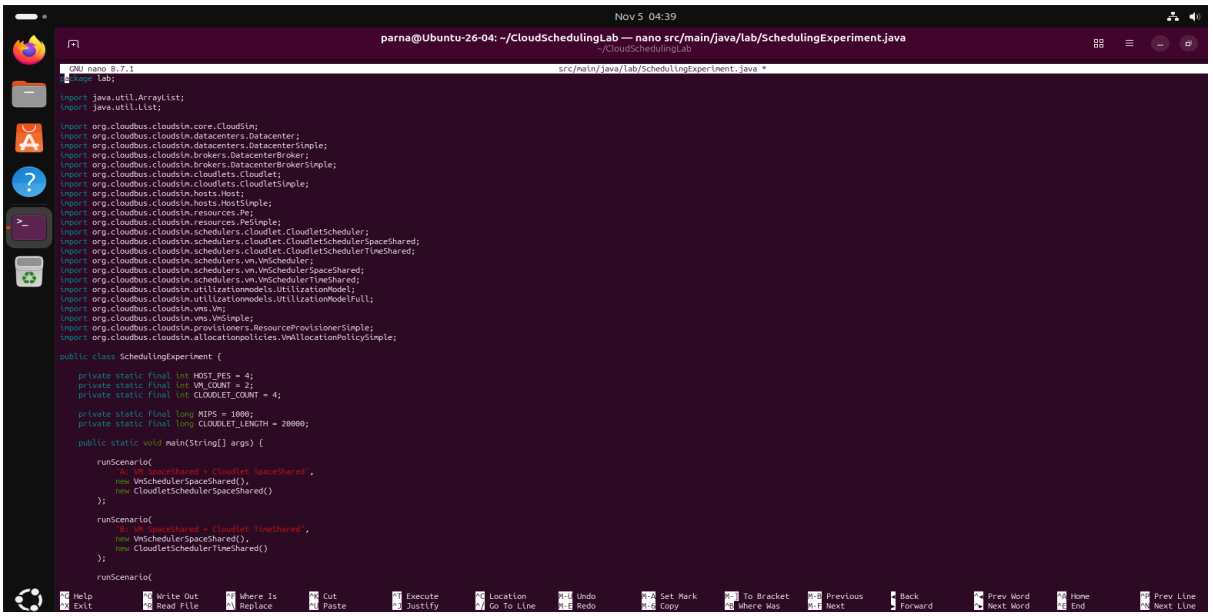
        for (int i = 0; i < CLOUDLET_COUNT; i++) {

            Cloudlet cloudlet = new CloudletSimple(
                CLOUDLET_LENGTH,
                1,
                utilization
            );

            cloudlet.setSizes(300);

            cloudletList.add(cloudlet);
        }

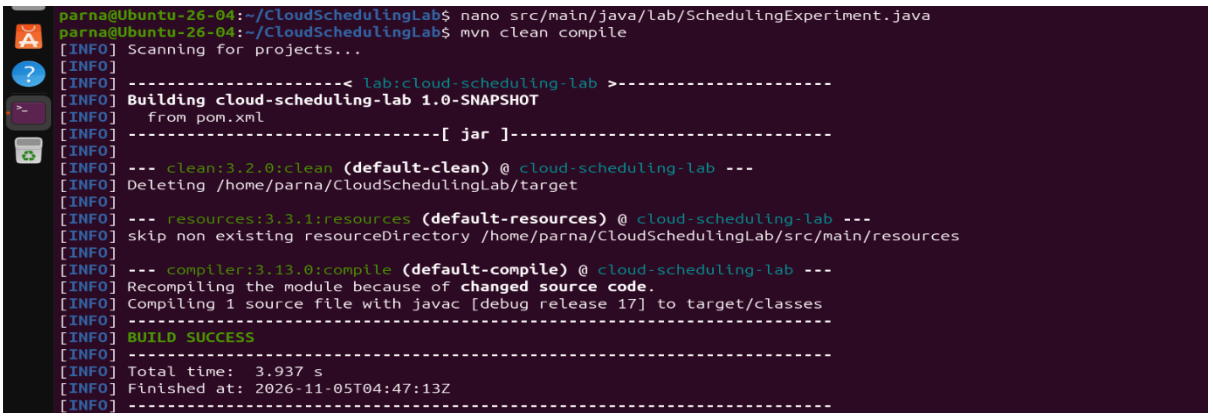
        return cloudletList;
    }
}
```



5. Compile and execute

5.1 Compile

mvn clean compile



Why: Compiles the Java source code and reports syntax or API errors before running the simulation.

5.2 Run

mvn exec:java -Dexec.mainClass="lab.SchedulingExperiment" -Dexec.classpathScope=compile

```
parna@Ubuntu-26-04:~/CloudSchedulingLab$ mvn exec:java -Dexec.mainClass="lab.SchedulingExperiment" -Dexec.classpathSc
ope=compile
INFO  CloudInformationService0: Notify all CloudSim Plus entities to shutdown.

INFO
================== Simulation finished at time 20.21 ==================

Cloudlet   VM         Status       Start        Finish
0          0          SUCCESS      0.10         20.10
2          0          SUCCESS      0.10         20.10
1          1          SUCCESS      0.10         20.10
3          1          SUCCESS      0.10         20.10
Completed Cloudlets: 4 / 4
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  1.874 s
[INFO] Finished at: 2026-11-05T04:53:11Z
[INFO] ------------------------------------------------------------------------
```

Why: Executes the main() method with Maven and the project dependency classpath.